

**FACULTAD DE INGENIERÍA - UNLP**  
**E1301 - E0301 Introducción a los Sistemas Lógicos y Digitales**  
**Curso 2025 - Trabajo Entregable 0**

*Grupo 01*

Ariza, Martin (02844/4 - [mar.ariza2003@gmail.com](mailto:mar.ariza2003@gmail.com))

Pascual Velazquez, Rocio (03377/1 - [rocio.pascual@alu.ing.unlp.edu.ar](mailto:rocio.pascual@alu.ing.unlp.edu.ar))

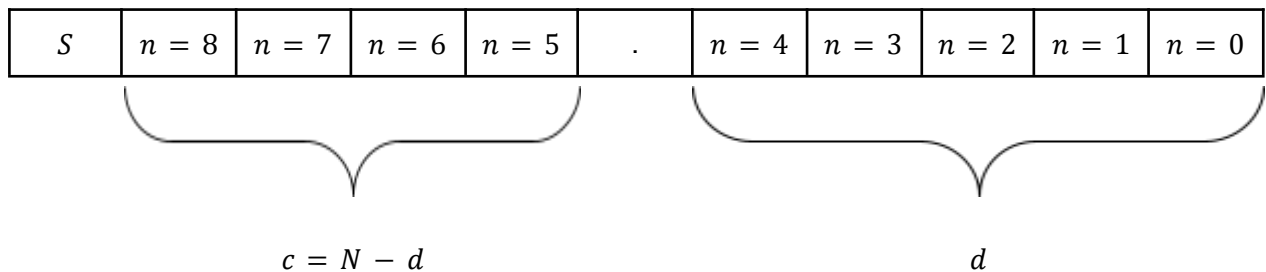
Guitera Woida, Emma (03352/2 - [emmagw04@gmail.com](mailto:emmagw04@gmail.com))

**Descripción**

Se sabe que la ecuación de una recta es:

$$y = mx + b$$

Sea  $Q(c, d)$ :



Resolución:  $2^{-d}$

Rango sin signo ( $S = 0$ ):

$$0 \leq x \leq 2^c - 2^{-d}$$

Rango con signo:

$$-2^{N-1-d} \leq x \leq 2^{N-1-d} - \frac{1}{2^d}$$

Representaciones:

- $m$ :  $Q(0, 15)$
- $b$ :  $Q(7, 8)$
- $x$ : *punto fijo* (32 bits)
- $y$ : *punto fijo* (32 bits)

a. ¿Cuál es el rango de representación y la resolución de los valores de  $m$ ?

Teniendo en cuenta lo explicado anteriormente:

$$N = 16, c = 0, d = 15 \rightarrow \text{escala: } 2^{-d} = 0.0000305$$

$$-2^{N-1-d} \leq x \leq 2^{N-1-d} - \frac{1}{2^d}$$

$$\text{rango: } -1 \leq x \leq 1 - 2^{-15}$$

$$\text{resolución: } 2^{-15}$$

b. ¿Cuál es el rango de representación y la resolución de los valores de  $b$ ?

Teniendo en cuenta lo explicado anteriormente:

$$c = 7, d = 8 \rightarrow \text{escala: } 2^{-8} = 0.00390625$$

$$\text{rango: } -2^7 \leq x \leq 2^7 - 2^{-8}$$

$$\text{resolución: } 2^{-8} = 0.00390625$$

c. Elija una representación  $Q(c, d)$  para  $x$  y para  $y$  tal que tanto  $m$  como  $b$  puedan representarse usando la misma representación sin pérdida de cifras significativas. Indique cuales serían los desplazamientos y/o máscaras que debe utilizar para ubicar las variables.

Para que tanto  $m$  y  $b$  se puedan representar sin pérdida de precisión:

- $m$  requiere 15 bits fraccionarios y 1 bit de signo.
- $b$  tiene 7 bits enteros y 8 bits fraccionarios.

Una opción que cubre ambos es  $Q(7, 15)$ , que da:

- 7 bits para parte entera (compatible con  $b$ )
- 15 bits para parte fraccionaria (compatible con  $m$ )

Para que la representación de  $x$  e  $y$  se utilicen 32 bits, como se indica en la del trabajo, se completó la parte real agregando bits en cero a la izquierda quedando la representación final de  $Q(16, 15)$

Observación

Desplazamientos y máscaras

- Para convertir un valor decimal a esta representación, se multiplica por  $2^{15}$
- Máscara para extraer solo la parte fraccionaria:  $0x7FFF$  (15 bits de parte fraccionaria). La máscara  $0x7FFF$  (que en binario es  $0111\ 1111\ 1111\ 1111$ ) se usa para extraer los 15 bits de la parte fraccionaria de una representación en punto fijo  $Q(7, 15)$ .

d. ¿Cuál va a ser el rango y la resolución de los valores que pueden representarse en x y en y?

Con Q(16, 15) (32 bits con signo):

- Rango:  $-2^{16} \leq x \leq 2^{16} - 2^{-15}$ 
  - Rango:  $-65536 \leq x \leq 65535.9999694824$
- Resolución:  $2^{-15} = 0.000030517578125$

e. De acuerdo con la ecuación de la recta y tomando casos límites respecto a los valores de las constantes m y b, por ejemplo el mínimo valor negativo o el máximo valor positivo, a qué valores debería acotarse x para que usando la representación elegida los valores de y obtenidos no produzcan overflow.

La ecuación de la recta es:

$$y = mx + b$$

- Mínimo valor de m: -1
- Máximo valor de m: 0.9999694824
- Mínimo valor de b: -128
- Máximo valor de b: 127.996093755
- Mínimo valor de y: -65536
- Máximo valor de y: 65535.9999694824

Para evitar overflow:

- $y \leq 65535.9999694824 \Rightarrow x \leq \frac{65535.9999694824 - 127.996093755}{0.9999694824}$
- $x \leq 65410.0000319443748$

Y para el valor más negativo:

- $y \leq -65536 \Rightarrow x \leq \frac{-65536 - 127.996093755}{0.9999694824}$
- $x \geq -65410.0000319443748$

f. Escriba un programa en lenguaje de programación C que permita el ingreso de un valor expresado en notación decimal ±eee.ffff y lo convierta a representación en punto fijo de 16 bits Q(7,8). El programa debe validar la entrada y determinar si el número ingresado está dentro del rango representable. La salida debe expresarse en Hexadecimal (0xHHHH)

### Descripción

El programa permite ingresar un número en notación decimal de la forma ±eee.ffff, donde ±:

- ± es el signo, indicando si el número es positivo o negativo.
- eee es la parte entera del número.
- ffff es la parte fraccionaria, que deberá completar con ceros si es necesario para asegurar el correcto funcionamiento del programa.

El programa valida que el número ingresado esté dentro del rango representable por Q(7,8), es decir, entre -128 y 127 (esto debido a que no se puede utilizar ningún tipo de dato float ni double por enunciado).

La validación se realiza multiplicando el número ingresado por 10.000, de manera que la parte fraccionaria sea manejada con precisión. Si el número está dentro de este rango, el programa continúa, de lo contrario, muestra un mensaje de error y termina.

El programa convierte el número decimal a su representación en punto fijo Q(7,8) de la siguiente manera:

- Parte entera: Se ajusta a un valor de 8 bits desplazando la parte entera a la izquierda (  $e \ll 8$  ).
- Parte fraccionaria: La parte decimal se convierte a su equivalente binario de 8 bits usando la fórmula  $((f * 256) / 10000)$  para escalar la fracción correctamente.
- El resultado final es la combinación de la parte entera y fraccionaria en la representación de punto fijo Q(7,8)

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VAL 127
#define MIN_VAL -128

int validarEntrada(char,int,int); // Prototipo de La funcion
int convertirAQ7_8(char,int,int); // Prototipo de La funcion

int main()
{
    char signo; int parteEntera, parteFraccionaria;

    printf("Ingrese un nro en notacion decimal ±eee.ffff: ");
    scanf(" %c%d.%d",&signo,&parteEntera,&parteFraccionaria);

    if (validarEntrada(signo,parteEntera,parteFraccionaria) == 1){
        printf("El nro ingresado esta dentro del rango representable\n");
    } else {
        printf("El nro ingresado no esta dentro del rango representable\n");
        return 1; // El programa no resulto exitoso
    }

    int valor = convertirAQ7_8(signo,parteEntera,parteFraccionaria);
    printf("Representacion en Q(7,8) en Hexadecimal: 0x%04X\n",(unsigned
short)valor);

    return 0; // El programa resulto exitoso
}

int validarEntrada(char s, int e, int f){
    int signo = 1,nro,resultado = 0; // Si resultado es 0 el nro no se encuentra
dentro del rango
    if (s == '-') {
        signo = -1; // Aplicar signo
    }
    nro = ((e * 10000) + f) * signo;
    if (nro >= (MIN_VAL*10000) && nro <= (MAX_VAL*10000)){
        resultado = 1; // Si resultado es 1 el nro esta dentro del rango
    }
}
```

```

    }
    return resultado;
}

int convertirAQ7_8(char s, int e, int f) {
    int signo = (s == '-') ? -1 : 1;
    // Convertimos la parte fraccionaria de base 10 a base 2 (Q8)
    int parteFraccionariaQ8 = (e << 8) + ((f * 256) / 10000);
    // Combinamos parte entera y fraccionaria
    int resultado = (e << 8) | parteFraccionariaQ8;
    return signo*resultado;
}

```

*Al hacer clic en este enlace, podrás descargar el código fuente del ejercicio*

[Grupo01f.c](#)

*g. Escriba otro programa que dado un número expresado en punto fijo de 16 bits Q(7,8) exprese el valor decimal equivalente eee.ffff. La entrada debe ser un número hexadecimal expresado en la forma 0xHHHH que debe ser validada.*

### Descripción

El programa permite ingresar un número hexadecimal expresado en formato Q(7,8), es decir, un número de 16 bits con la siguiente estructura:

- **0xHHHH**: Número hexadecimal con signo (16 bits)

El programa valida que el número hexadecimal ingresado esté dentro del rango representable por Q(7,8). La validación se realiza asegurando que el valor de la parte entera, obtenida al dividir el número por 256, se encuentre dentro del rango de -128 a 127.

El programa extrae las siguientes dos partes del número hexadecimal:

- Parte entera: Se obtiene desplazando el valor del número a la derecha en 8 bits. Esta parte se ajusta a un valor de 8 bits.
- Parte fraccionaria: Se extrae mediante una operación de máscara (AND con 0xFF), y luego se convierte a su equivalente decimal usando la fórmula  $((\text{fracción} * 10000) / 256)$  para obtener la parte decimal con precisión.

Si el número es negativo, se ajusta el valor de la parte fraccionaria en complemento a dos para reflejar correctamente su valor decimal.

Finalmente, el programa imprime el número convertido en formato decimal con 3 dígitos en la parte entera y 4 dígitos en la parte fraccionaria.

```

#include <stdio.h>
#include <stdint.h>

int main() {
    int16_t hex_value;

    // Pedimos el número en hexadecimal
    printf("Ingrese un número hexadecimal (ej: 0xFE5C): ");
    scanf("%hx", &hex_value); // %hx para Leer un número hexadecimal de 16 bits con signo

    // Extraer la parte entera
    int16_t entero = hex_value / 256;

    // Extraer la parte fraccionaria (8 bits inferiores)
    int16_t fraccion_binaria = hex_value & 0xFF;

    // Convertir la parte fraccionaria a 4 dígitos decimales
    int16_t fraccion = (fraccion_binaria * 10000) / 256;

    // Ajuste si el número es negativo (complemento a dos)
    if (hex_value < 0 && fraccion_binaria != 0) {
        fraccion = 10000 - fraccion; // Complemento de la parte fraccionaria
    }

    // Imprimimos el resultado en formato decimal con 3 decimales
    if (entero < -128 || entero > 127) {
        printf("Error: El número no se puede representar en formato Q(7,8).\n");
        return 1;
    }
    // Imprimir resultado con 3 dígitos en la parte entera y 4 en la fraccionaria
    printf("El valor en Q(7,8) es: %d.%04d\n", entero, fraccion);

    return 0;
}

```

Al hacer clic en este enlace, podrás descargar el código fuente del ejercicio

[Grupo01g.c](#)

*h. Escriba un programa que permita el ingreso de los valores de  $m$ ,  $b$  y  $x$  en forma decimal y muestre su representación en punto fijo en formato hexadecimal validando la entrada como en los puntos anteriores. Luego realizando todas las operaciones en punto fijo con las representaciones adoptadas calcule el valor de la ordenada y  $y$  lo muestre en punto fijo en forma hexadecimal y en decimal.*

El programa permite ingresar 3 números en notación decimal de la forma  $\pm eee.ffff$ , entre ellas:

- $m$ : pendiente de la función
- $x$ : variable de la que depende  $y$
- $b$ : ordenada al origen

Una vez que se ingresa cada decimal, se muestra su representación en punto fijo en formato hexadecimal a través de las funciones creadas en el inciso f.

Luego se obtiene  $y$  a través del siguiente cálculo:

$$y = m \cdot x + b$$

Al hacer  $y = m \cdot x$  se realiza un desplazamiento de 15 bits para que se mantenga  $Q(16,15)$  y se realice la operación en la posición correcta de cada bit.

A ese resultado se le suma  $b$ , por lo que se obtiene  $y$  en hexadecimal.

Posterior a eso se hace la transformación a decimal para mostrar el resultado en decimal utilizando las funciones creadas en el inciso g.

```
#include <stdio.h>
#include <stdint.h>

#define MAX_Q7_8 127
#define MIN_Q7_8 -128
#define MAX_Q0_15 0.999969 // para orientación

int ejerF(char);
int validarEntrada(char, int, int);
int convertirAQ7_8(char, int, int);
int convertirAQ0_15(char, int, int);
int convertirAQ16_15(char, int, int);
int ejerG(int);

int main() {
    int m_q15 = ejerF('m'); // Q(0,15)
    if (m_q15 == 1) return 1;

    int b_q7_8 = ejerF('b'); // Q(7,8)
    if (b_q7_8 == 1) return 1;

    int x_q16_15 = ejerF('x'); // Q(16,15)
    if (x_q16_15 == 1) return 1;

    // Multiplicación: m * x (Q(0,15) * Q(16,15) = Q(16,30))
    int64_t producto = (int64_t)m_q15 * x_q16_15;

    // Ajuste de escala: Q(16,30) >> 15 → Q(16,15)
    int32_t mx_q16_15 = (int32_t)(producto >> 15);

    // Convertir b (Q(7,8)) a Q(16,15): b << 7
```

```

    int32_t b_convertido = b_q7_8 << 7;

    // Sumar y obtener y en Q(16,15)
    int32_t y_q16_15 = mx_q16_15 + b_convertido;

    printf("y = 0x%08X\n", (uint32_t)y_q16_15);
    ejerG(y_q16_15);

    return 0;
}

// Ingreso de datos decimal ±eee.ffff
int ejerF(char param) {
    char signo;
    int parteEntera, parteFraccionaria;
    int valor;

    printf("Ingrese %c en notacion decimal ±eee.ffff: ", param);
    scanf(" %c%d.%d", &signo, &parteEntera, &parteFraccionaria);

    if (param == 'm') {
        valor = convertirAQ0_15(signo, parteEntera, parteFraccionaria);
        printf("Representación en Q(0,15): 0x%08X\n", valor);
    } else if (param == 'b') {
        if (!validarEntrada(signo, parteEntera, parteFraccionaria)) {
            printf("Error: b fuera del rango Q(7,8)\n");
            return 1;
        }
        valor = convertirAQ7_8(signo, parteEntera, parteFraccionaria);
        printf("Representación en Q(7,8): 0x%04X\n", (uint16_t)valor);
    } else {
        valor = convertirAQ16_15(signo, parteEntera, parteFraccionaria);
        printf("Representación en Q(16,15): 0x%08X\n", valor);
    }

    return valor;
}

// Valida entrada en rango Q(7,8)
int validarEntrada(char s, int e, int f) {
    int signo = (s == '-') ? -1 : 1;
    int nro = ((e * 10000) + f) * signo;
    return (nro >= (MIN_Q7_8 * 10000) && nro <= (MAX_Q7_8 * 10000));
}

// Conversiones
int convertirAQ7_8(char s, int e, int f) {
    int signo = (s == '-') ? -1 : 1;
    int decimal = (e << 8) + ((f * 256 + 5000) / 10000);
    return signo * decimal;
}

```

```

int convertirAQ0_15(char s, int e, int f) {
    if (e != 0) return 0; // en Q(0,15) no se permiten enteros
    int signo = (s == '-') ? -1 : 1;
    return signo * ((f * 32768 + 5000) / 10000);
}

int convertirAQ16_15(char s, int e, int f) {
    int signo = (s == '-') ? -1 : 1;
    return signo * ((e << 15) + ((f * 32768 + 5000) / 10000));
}

// Mostrar en notación decimal ±eee.ffff
int ejerG(int valor) {
    int negativo = (valor < 0);
    if (negativo) valor = -valor;

    int parte_entera = valor >> 15;
    int fraccion = ((valor & 0x7FFF) * 10000 + 16384) / 32768;

    if (negativo) printf("El valor en Q(16,15) es: -%.%04d\n", parte_entera, fraccion);
    else printf("El valor en Q(16,15) es: %.%04d\n", parte_entera, fraccion);

    return 0;
}

// Prueba de escritorio para y = m * x + b
// Valores de entrada
m = +0.0001 → Q(0,15): 0x00000003
b = +1.0000 → Q(7,8): 0x0100
x = -1.0000 → Q(16,15): 0xFFFF8000
// Salida
y = +0.9999 → Q(16,15): 0x00007FFD

```

Al hacer clic en este enlace, podrás descargar el código fuente del ejercicio  
[Grupo01hReentrega.c](http://Grupo01hReentrega.c)